

Software Engineering (STE)

Course Code : 315323

Lecturer – Asad Shamna

Semester – 5th

I. RATIONALE

Software Engineering is the foundation for professional processes to be followed for designing, developing, testing and maintaining software involving principles, different techniques, and practices for software development. This course enable students to develop requisite abilities to follow systemic and disciplined approach to software development that aims to create high quality, reliable and maintainable software.

II. INDUSTRY / EMPLOYER EXPECTED OUTCOME

Apply software engineering principles to develop software product.

III. COURSE LEVEL LEARNING OUTCOMES (COS)

Students will be able to achieve & demonstrate the following COs on completion of course based learning

CO1 – Select suitable software development process model

CO2 – Prepare software requirement specification.

CO3 – Construct different Software design models

CO4 – Apply different planning and cost estimation techniques for a software product

CO5 – Apply project management techniques in software development.

CO6 – Use quality assurance principles in software development

UNIT - II SOFTWARE REQUIREMENT ENGINEERING [14M]

2.1 Software Engineering core principles.

2.2 Software Practices: Communication, Planning, Modelling, Construction, Software deployment (Statement and meaning of each principles for each practice).

2.3 Requirement Engineering: Requirement Gathering and Analysis, Types: Functional, Product, organizational, External Requirements, Eliciting Requirements, Developing Use-cases, Building requirement models, Negotiation, Validation.

2.4 Software Requirement Specification: Need, Format, and its Characteristics.

2.1 Software Engineering:

- Software engineering is a discipline that applies engineering principles to the design, development, testing, and maintenance of software systems. It's a systematic and structured approach focused on creating high-quality, reliable, and efficient software solutions.
- This field is broader than just programming; it encompasses the entire software lifecycle, including requirements analysis, design, coding, testing, and maintenance, often for large-scale projects.

2.1 Software Engineering core principles.

- 1. Modularity:** Breaking down the software into smaller, independent, and reusable components or modules. This makes the software easier to understand, test, and maintain.
- 2. Abstraction:** Hiding the implementation details of a module or component and exposing only the necessary information. This makes the software more flexible and easier to change.
- 3. Encapsulation:** Wrapping the data and functions of a module or component into a single unit, and providing controlled access to that unit. This helps to protect the data and functions from unauthorized access and modification.
- 4. DRY principle (Don't Repeat Yourself):** Avoiding duplication of code and data in the software. This makes the software more maintainable and less error-prone.
- 5. KISS principle (Keep It Simple, Stupid):** Keeping the software design and implementation as simple as possible. This makes the software more understandable, testable, and maintainable.

2.1 Software Engineering core principles.

6. YAGNI (You Ain't Gonna Need It): Avoiding adding unnecessary features or functionality to the software. This helps to keep the software focused on the essential requirements and makes it more maintainable.

7. SOLID principles: A set of principles that guide the design of software to make it more maintainable, reusable, and extensible. This includes the Single Responsibility Principle, Open/Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, and Dependency Inversion Principle.

8. Test-driven development: Writing automated tests before writing the code, and ensuring that the code passes all tests before it is considered complete. This helps to ensure that the software meets the requirements and specifications.

2.2 Software Practices:

2.2.1 Communication:

- Effective communication is crucial in software engineering, impacting everything from project success to team dynamics and individual performance. Key practices include active listening, concise and clear messaging, choosing the right communication methods, and fostering a culture of feedback.
- Software engineers should also be adapt at communicating technical concepts to both technical and non-technical audiences.

Key Communication Practices in Software Engineering:

- **Active Listening:** Pay close attention to what others are saying, both verbally and nonverbally, to understand their perspectives and needs.
- **Clear and Concise Messaging:** Use simple language, avoid jargon, and structure your communication to be easily understood.
- **Choose the Right Medium:** Select the most appropriate communication channel (e.g., email, instant messaging, video calls, face-to-face meetings) based on the context and audience.

Key Communication Practices in Software Engineering:

- **Feedback Culture:** Provide and receive constructive feedback regularly to improve individual and team performance.
- **Active Participation:** Engage actively in discussions, meetings, and brainstorming sessions.
- **Preparation:** Before communicating, especially in meetings, prepare talking points and gather information to ensure clarity and effectiveness.
- **Visual Aids:** Use diagrams, charts, and other visual aids to explain complex technical concepts.
- **Documentation:** Keep records of decisions, discussions, and agreements to avoid misunderstandings and ensure everyone is on the same page.
- **Facilitation:** In group settings, have a facilitator to guide discussions, manage time, and ensure everyone has a chance to contribute.
- **Understanding the Audience:** Tailor your communication style and language to the specific audience, whether technical or non-technical.

Key Communication Practices in Software Engineering:

- **Conflict Resolution:** Use communication skills to address conflicts constructively and find mutually agreeable solutions.
- **Regular Meetings:** Establish a regular cadence for team meetings (e.g., daily stand-ups, sprint planning) to maintain alignment and address issues promptly.
- **Open Communication:** Encourage open and honest communication to foster trust and collaboration within the team.
- **Empathy:** Demonstrate empathy and understanding when communicating with others, considering their perspectives and emotions.

Benefits of Effective Communication:

- **Improved Collaboration:** Clear communication enables teams to work together more effectively, sharing ideas, and solving problems collaboratively.
- **Reduced Errors and Increased Efficiency:** When everyone understands their roles and responsibilities, errors are minimized and productivity increases.
- **Enhanced Problem Solving:** Effective communication facilitates open discussions, allowing teams to identify and address problems more efficiently.
- **Stronger Team Dynamics:** Open communication fosters a positive and collaborative team environment, where everyone feels comfortable sharing ideas and contributing.
- **Better Project Outcomes:** Clear communication contributes to better project planning, execution, and delivery.

2.2.2 Planning:

- Effective planning in software engineering involves a series of practices focused on defining project scope, identifying risks, allocating resources, and establishing a development process.
- Key practices include understanding requirements, creating a detailed project plan, managing risks, defining architecture, establishing a development process, and ensuring effective communication.
- These practices contribute to a more organized and successful software development lifecycle.

Key Planning Practices:

- **Define Project Scope:** Clearly outline the objectives, deliverables, and boundaries of the software project.
- **Risk Management:** Identify potential risks, assess their impact, and develop mitigation strategies.
- **Resource Allocation:** Determine the necessary human resources, technology, and infrastructure for the project.
- **Development Process:** Establish a structured approach for development, including methodologies, tools, and standards.
- **Testing Strategy:** Define the testing approach, including types of tests and schedule.
- **Communication Plan:** Ensure clear and effective communication channels among team members and stakeholders.
- **Iterative Planning:** Recognize that planning is an ongoing process and adjust the plan as needed.
- **Customer Involvement:** Engage the customer throughout the planning phase to ensure alignment with their needs.
- **Quality Assurance:** Implement procedures to ensure the software meets quality standards and requirements.

Benefits of Effective Planning:

- **Reduced risks and uncertainties:** Proactive planning helps anticipate and mitigate potential problems.
- **Improved resource utilization:** Efficient planning ensures resources are allocated effectively.
- **Enhanced project control:** A well-defined plan provides a roadmap for development and helps track progress.
- **Increased team efficiency:** Clear roles, responsibilities, and communication contribute to a more productive team.
- **Higher quality software:** A focus on quality throughout the planning and development process leads to a better product.
- **Better alignment with customer needs:** Involving the customer in planning ensures the final product meets their expectations.

2.2.3 Modelling:

In software engineering, modeling refers to creating abstract representations (models) of a software system to aid in understanding, communication, and analysis of its structure, behavior, and relationships before actual coding begins. These models serve as blueprints, helping developers, designers, and stakeholders visualize and plan the software, manage complexity, and make informed decisions.

Purpose of Software Modeling

Understanding and Communication: Models simplify complex systems, making them easier to understand and discuss among various stakeholders (developers, users, clients, etc.).

Analysis and Design: Models allow for analysis of the system's behavior, performance, and potential issues before implementation, helping identify potential problems early on.

Guiding Development: Models act as a guide during the development process, ensuring that the final product aligns with the intended design and requirements.

Facilitating Collaboration: Models promote better communication and collaboration among the development team and other stakeholders by providing a shared understanding of the system.

Types of Software Models:

- **Data Models:** Represent the structure and relationships of data within the system (e.g., Entity-Relationship Diagrams).
- **Behavioral Models:** Describe how the system behaves and responds to events (e.g., State Diagrams, Sequence Diagrams).
- **Architectural Models:** Illustrate the overall structure and organization of the software system.
- **User Interface (UI) Models:** Represent the user interface and how users interact with the system.

2.2.4 Construction:

- **Modularity and Separation of Concerns:** Break down software into smaller, independent modules, each handling a specific concern, to improve maintainability and scalability.
- **Code Quality:** Write clear, readable, and well-documented code, using meaningful variable and function names.
- **Unit Testing:** Create and apply a set of unit tests for each component to verify its functionality as it is written.
- **Code Reviews:** Involve a team to review code for correctness, logic, readability, and adherence to standards.
- **Readability and Simplicity:** Keep conditional logic as simple as possible and use consistent naming conventions to enhance code clarity.

2.2.5 Software deployment:

Software deployment is the process of making a software application available for use. It encompasses a series of activities, from preparing the software for release to installing and configuring it on target systems.

- **Fixing bugs and Enhancing Performance:** The release of updates, bug fixes, and performance enhancements is made possible by deployment. It guarantees that consumers are utilizing reliable and effective software versions.
- **Competitiveness and Time-to-Market:** Time-to-market is shortened by quick and effective deployment, which helps businesses remain competitive in ever-changing markets. Having the capacity to release updates rapidly can give you a competitive edge.
- **User Experience and Satisfaction:** Updates that are regularly released guarantee that consumers receive the newest enhancements, which raises user happiness. Promptly implementing deployment in response to customer input improves the overall user experience.

2.2.5 Software deployment:

- **Feedback System for Ongoing Improvement:** Frequent deployment establishes a feedback loop that offers insightful information about how consumers engage with the programme. This information can be utilized to guide future development and deployment strategies and promote continual improvement.
- **Optimizing Resources and Cutting Costs:** Optimizing resources and cutting costs are two benefits of efficient deployment techniques. Processes for deployment can be automated to save manual labor, cut down on mistakes, and guarantee consistent outcomes in all situations.

2.3 Requirement Engineering:

2.3.1 Requirement Gathering and Analysis:

Requirement gathering and analysis is the crucial first step in the Software Development Life Cycle (SDLC) (SDLC) where stakeholders needs and expectations are systematically collected, understood, and documented to form a clear vision for a product or system. This process involves eliciting **functional (what the system does) and non-functional requirements (how it performs)** through techniques like interviews and focus groups, then analyzing them for issues like ambiguity, inconsistency, and incompleteness to ensure a shared understanding and prevent costly changes later in the project.

Types: Functional, Product, Organizational, External Requirements, Eliciting Requirements, Developing Use-cases, Building requirement models, Negotiation, Validation.

1) Functional: In software engineering, "functional" refers to the specific features and behaviors a system must exhibit to perform its intended tasks and meet user needs, detailing what the software should do. These are also known as functional requirements and describe actions like data processing, user interactions, calculations, and other specific operations, forming the core functionality that developers build the system around.

2) Product: Product Requirements Document (PRD) is a fundamental aspect of product management, which plays a critical role in the development of successful products. Product requirements in software engineering are high-level descriptions of a product's desired features, functions, and behaviors, defining what the product should do to meet user and business goals, rather than how it should be implemented.

3) Organizational: Organizational requirements in software engineering are the needs, policies, and objectives of a business that the software must satisfy to support its operations effectively, encompassing aspects like business goals, compliance, integration, security, scalability, and user experience.

4. External Requirements: In software engineering, external requirements define aspects that originate outside the system itself but are necessary for its proper interaction with its environment and stakeholders. They typically encompass external interfaces, such as user, software, and communication interfaces, which dictate how the system interacts with users, other applications, and external hardware or networks.

5. Eliciting Requirements: The process of investigating and learning about a system's requirements from users, clients, and other stakeholders is known as requirements elicitation. Requirements elicitation in software engineering is perhaps the most difficult, most error-prone, and most communication-intensive software development.

6. Developing Use-cases: Developing use cases in software engineering involves creating detailed descriptions of how a system interacts with its users or other external entities to achieve specific goals. This process is crucial for capturing functional requirements and ensuring the developed software meets user needs.

2.3.6 Steps for Developing Use Cases:

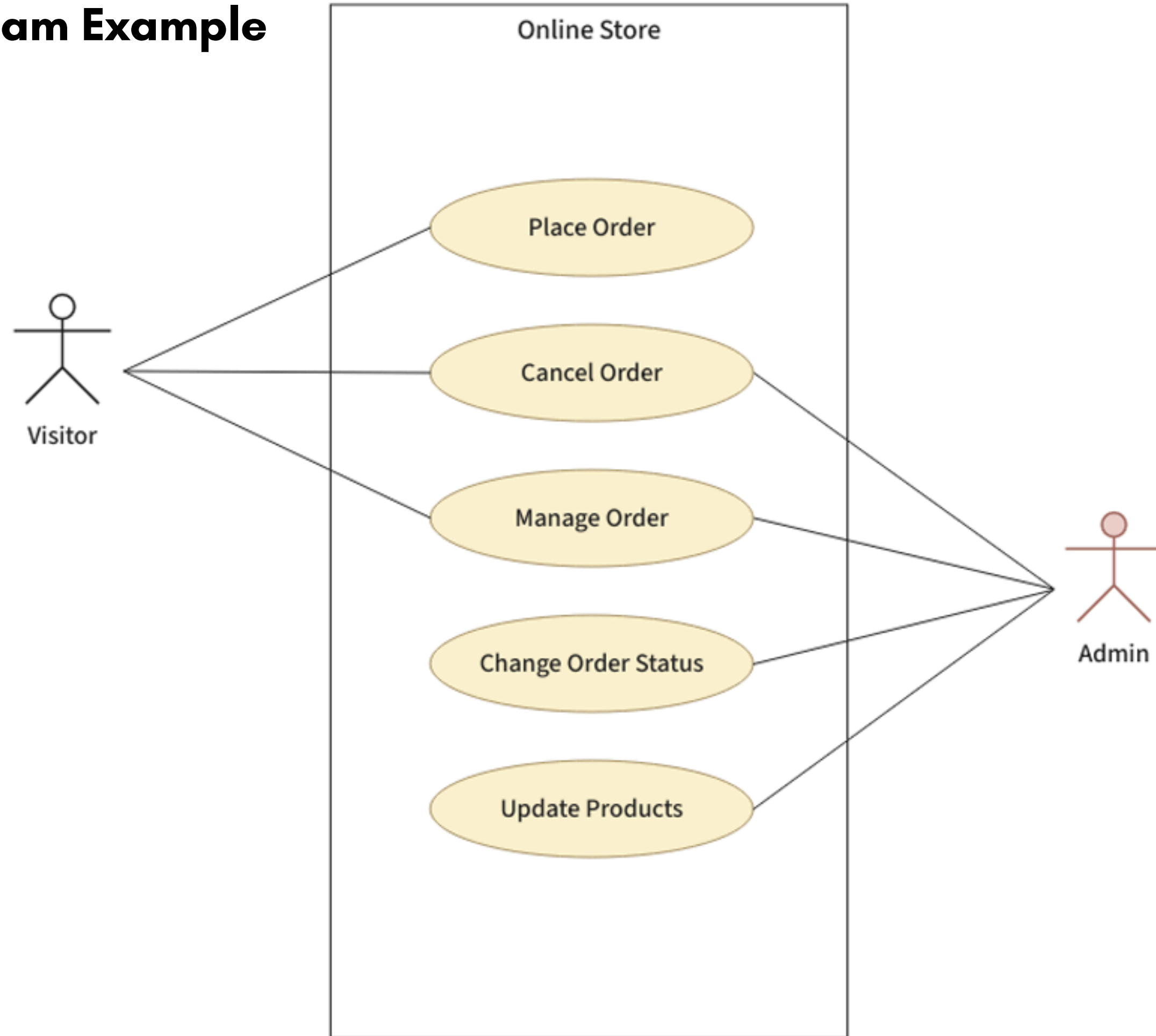
Identify Actors: Determine all entities that interact with the system. Actors can be human users, other systems, or even time-based events. Categorize them as primary actors (initiating interactions) and secondary actors (responding to primary actors).

Identify Use Cases: For each actor, identify the specific goals they want to achieve through interacting with the system. Each goal typically corresponds to a distinct use case. Use cases should be named clearly and concisely, reflecting the user's objective (e.g., "Register for Course," "Process Order").

Define Use Case Descriptions: For each identified use case, create a detailed description outlining the sequence of events. This typically includes:

- **Use Case Name:** A clear and concise name.
- **Actors:** The primary and secondary actors involved. (Actors are referred as Stakeholders)
- **Preconditions:** Conditions that must be true before the use case can begin.
- **Trigger:** The event that initiates the use case.
- **Basic Flow (Main Scenario):** The ideal, successful sequence of steps from the actor's perspective.
- **Alternate Flows (Extensions):** Variations or alternative paths that can occur, including error handling or optional steps.
- **Postconditions:** The state of the system after the use case is successfully completed.

Use Cases Diagram Example



2.3.7 Building requirement models:

Building requirement models involves creating structured, often graphical, representations of a software system's functional and non-functional requirements to ensure clarity for all stakeholders. This process uses various techniques, including Use Case diagrams for system interactions, User Stories for high-level goals, Activity Diagrams for process flows, and Class Diagrams for static structure, all contributing to a shared understanding of the system's behavior and constraints before development begins.

Key Steps in Building a Requirements Model

1. Requirements Elicitation: Gather requirements from stakeholders through interviews, workshops, and other methods to understand their needs.

2. Modeling Requirements:

Apply various modeling techniques to represent these requirements from different perspectives:

- **Scenario-Based Modeling:** Focuses on how users interact with the system.
- **Use Case Diagrams:** Show the interactions between users (actors) and the system's functionalities.
- **User Stories:** Short, simple descriptions of a feature told from the perspective of the user or customer.

2.3.7 Building requirement models:

2. Modeling Requirements:

CONTINUED

- **Data Modeling:** Represents the structure of data the system will handle.
- **Entity-Relationship (ER) Diagrams:** Illustrate entities and their relationships.
- **Flow-Oriented Modeling:** Describes the flow of data and processes within the system.
- **Data Flow Diagrams (DFDs):** Show how data moves through the system.
- **Activity Diagrams:** Depict the flow of activities in a process or use case.
- **Behavioral Modeling:** Illustrates how the system's behavior changes in response to events.
- **State Diagrams:** Show different states an object can be in and the transitions between them.
- **Class-Based Modeling:** Focuses on the classes, their attributes, and their relationships.
- **Class Diagrams:** Use in Object-Oriented Analysis to show classes, their attributes, operations, and relationships.

2.3.7 Building requirement models:

3. **Documentation:** Supplement diagrams with written documentation, including functional and non-functional requirements.
4. **Validation:** Review the models with stakeholders to ensure they accurately reflect the project's goals and constraints.
5. **Management:** Continuously manage changes and updates to the models as the project progresses.

2.3.8 Negotiation:

This step involves reaching a consensus on conflicting requirements from different stakeholders. The goal is to prioritize and resolve discrepancies to form a single, unified set of requirements.

2.3.9 Validation:

The documented requirements are reviewed and verified to ensure they are correct, complete, and meet the needs of the stakeholders.

2.4 Software Requirement Specification:

A **Software Requirements Specification (SRS)** format includes sections like **Introduction** (Purpose, Scope), **Overall Description** (Product Perspective, User Characteristics), **Specific Requirements (Functional, Non-Functional)**, and **Appendices**. Key characteristics of a good SRS are that it must be **complete, correct, consistent, unambiguous, verifiable, modifiable, and traceable**. It also should be **clear, testable, and design-independent**, serving as a binding contract between the customer and development team.

2.4.1 Need:

Clear Communication: An SRS provides a common understanding for all parties involved, including clients, developers, and testers.

Reduces Errors and Rework: By capturing requirements in detail upfront, it minimizes misunderstandings, leading to less rework during development.

2.4.1 Need:

Sets Expectations: It serves as a formal agreement on the project's scope, objectives, and the features the final software must possess.

Foundation for Development: The SRS guides the design and implementation process, ensuring that the development team builds the correct product.

Basis for Testing: Testing teams use the SRS to create comprehensive test cases, ensuring that the software meets the specified requirements.

Cost and Schedule Control: A well-defined SRS helps in accurately estimating project costs, risks, and timelines.

Reference for Maintenance: It provides a historical record and reference point for future enhancements, maintenance, and updates.

2.4.2 Format:

A Software Requirements Specification (SRS) follows the **IEEE 830 standard format, outlining the software's purpose, features, user interactions, and technical constraints** to guide the development process. The format typically includes sections for the Introduction, which details the purpose and scope; an Overall Description of the product and its constraints; and Specific Requirements, which encompass functional, non-functional, and external interface requirements, often supported by user stories or use cases.

Purpose of this Document - At first, main aim of why this document is necessary and what's purpose of document is explained and described.

Scope of this document - In this, overall working and main objective of document and what value it will provide to customer is described and explained. It also includes a description of development cost and time required.

Overview - In this, description of product is explained. It's simply summary or overall review of product.

2.4.2 Format: Standard IEEE 830 SRS Outline

1. Introduction

1.1 Purpose: Define the goals and intended audience of the software.

1.2 Scope: Explain what the software will (and will not) do, including benefits and objectives.

1.3 Definitions, Acronyms, and Abbreviations: Clarify special terms used in the document.

1.4 References: List related documents, parent projects, or standards.

1.5 Overview: Summarize the rest of the SRS document.

2. Overall Description

2.1 Product Perspective: Describe how the product relates to other systems, previous software, or operating environments.

2.2 Product Functions: Summarize the major actions or tasks the software will perform.

2.3 User Classes and Characteristics: Identify who will use the system (e.g., admin, regular user, guest) and their technical skill levels.

2.4.2 Format: Standard IEEE 830 SRS Outline

2. Overall Description

2.4 Operating Environment: Define the hardware, software, and operating system constraints.

2.5 Design and Implementation Constraints: List any limits imposed on developers (e.g., specific regulatory compliance or database types).

2.6 Assumptions and Dependencies: Note external factors that could affect project success.

3. Specific Requirements: (The core technical section using "shall" statements to define exact behaviors)

3.1 External Interface Requirements:

- User interfaces
- Hardware interfaces
- Software interfaces
- Communications interfaces

2.4.2 Format: Standard IEEE 830 SRS Outline

3. Specific Requirements:

3.2 Functional Requirements: Define fundamental actions, inputs, data processing, and resulting outputs.

3.3 Performance Requirements: Specify speed, capacity, response times, and throughput limits.

3.4 Logical Database Requirements: Define data types, frequency of use, and data retention rules.

3.5 Design Constraints: Specify code standards or modular requirements.

3.6 Software System Attributes: Cover non-functional properties like reliability, availability, security, and maintainability.

4. Appendices: A section reserved for supplementary information that is too detailed, bulky, or technical to include in the main body of the document.

5. Index: An alphabetical list of key terms, functions, modules, and concepts used throughout the document, along with their corresponding page numbers.

2.4.3 Characteristics:

A Software Requirements Specification (SRS) should be Complete, Consistent, Unambiguous, Traceable, Verifiable, and Modifiable. Key characteristics include accuracy to customer needs, clear documentation for all stakeholders, and the ability to be tested and updated without affecting other parts of the system. The SRS serves as a binding agreement for the project's scope and expectations, guiding development, testing, and maintenance.

Here are the key characteristics of an effective SRS:

Complete: The document should include all functional and non-functional requirements, system constraints, and external interfaces, leaving no essential information or features absent.

Correct: It must accurately reflect the actual needs and specifications of the customer and stakeholders.

Unambiguous: Every requirement should have only one clear interpretation, avoiding vague language or confusing statements.

Consistent: The SRS should be free of contradictions between requirements, and it should maintain uniform terminology and formatting throughout the document.

2.4.3 Characteristics:

Here are the key characteristics of an effective SRS:

Verifiable:

Each requirement must be testable, meaning there's a way to quantifiably measure if the final software meets that requirement.

Traceable:

Every requirement should have a unique identifier and an easily traceable origin and relationship to other requirements and system components.

Modifiable:

The SRS should be structured to allow for easy modification and updating of requirements as the project progresses without disrupting other parts of the document.

Prioritized (or Ranked):

Requirements should be ranked based on their importance and stability, helping to define the order in which they need to be fulfilled.